ELEC-H-310
Th01: C Language

Dragomir Milojevic
Université libre de Bruxelles

v3.3

Today

1. C language: why & how
2. A very first tour of C – functions
3. Data types and variables
4. Control structures
5. Loops

1. C language: why & how

What is C is (very) important

- C is a **general purpose** programming language widely used in many different applications; serves as a base for the development of many other languages (C++, Rust, etc.)
- C is **universal programming language** – could be used to program SW for any computer architecture including super-computers, high-end, low-end desktops/laptops, tablets, smartphones and also low-end embedded computers all around us
- Developed by Dennis Ritchie & Ken Thompson (1972), Bell Labs (USA) in efforts associated to the development of UNIX operating system (one serious OS out there) and as evolution of language B introduced few years earlier

Main characteristics

- C is considered to be low-level & universal
  - Low level – C is close to the machine and HW, so you can expect programs to be efficient in execution; C is compiled language, that is translated to native machine language; this is very much different from interpreted languages (Python, Java, etc.) that must use an intermediate layer of SW to insure instruction interpretation at run-time and thus introduce inefficiencies (lower performance / watt)
  - Universal – could be used to program SW for super-computers, high-end & low-end desktops/laptops but also embedded systems
- Being low-level makes mapping of the C code into assembly (machine code) simpler; compilers are easy to write and will be able to produce efficient machine code
- C programs could be independent from a specific computer architecture and the OS; if you write a C program using AINSI-C (standardized language version), it will compile on any machine/compiler couple (at least in theory)

Other (VERY) important characteristics

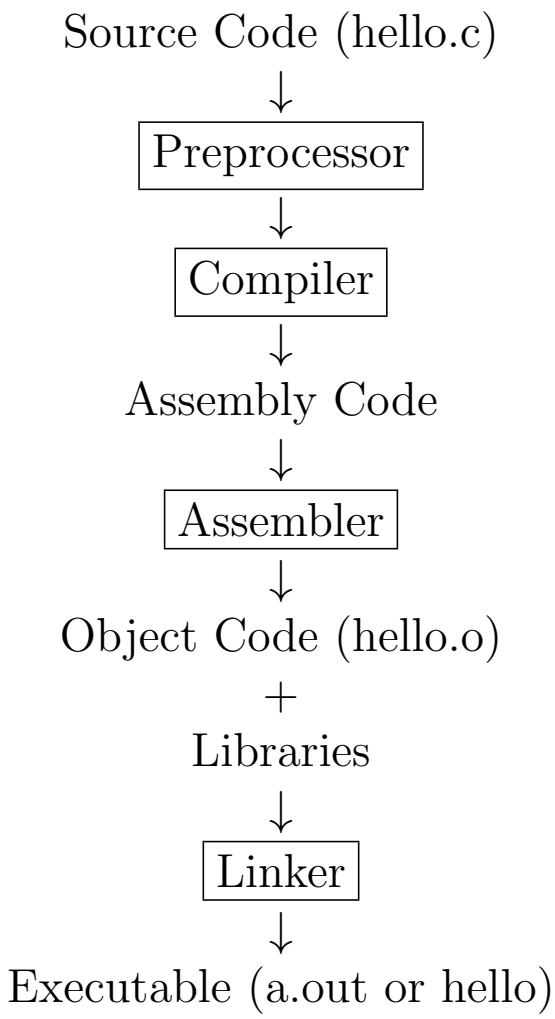
- Provides various mechanisms to manage the control of instruction flow, allowing structured programming for better programming efficiency through code readability, shared work and program maintenance
  - Structured programming is enabled in bottom-up fashion with: loops, conditions, code blocks within functions, and C functions, sub-programs & libraries for complex program management
- C is strongly-typed meaning that data used will need very precise definition; this allows SW to closely couple to HW (e.g. we have seen fixed point representation as opposed to floating point)
- Use of pointers (addresses) – very powerful mechanism that allows data, so memory, manipulation in a very efficient way; thanks to pointers certain algorithms yield programs that have excellent execution times

Why learning C today?

- Despite many different computer languages developed since '70, C still remains interesting & doesn't seam to age
- It is the language of choice for system level programming (OS, device drivers etc.), but is also used for application programming
- We said it is low-level, but it does provide a good enough support for abstraction (high-level) $\rightarrow$ C has nice balance between the two
- It is a language of choice where performance and efficiency are crucial; enables architecture dependent software optimizations – takes into account computer architecture to produce better code
  - Architecture dependent optimizations are done at the expense of portability $\rightarrow$ portability & efficiency are in general orthogonal
- Some believe that it is essential for an engineer to learn how to program in C and to understand limits of compilers

Life of a "C" program

- Starts with source code, a text file written using a text editor
  - ▹ Advice! Learn how to use a good text editor (e.g. VIM); learning curve may not be steep, but you will gain a lot of time afterwards due to features that typical GUI editors don't have
- Program is analyzed by pre-processor & then compiled into assembly code (human readable) and then converted into object file (binary for a given C module) using assembler
- Multiple object files (and eventually libraries) are linked using yet another SW (linker) to create the executable file that can be launched through OS



2. A very first tour of C – functions

"Hello world" – simplest C program

- Code below is **complete** – it contains all ingredients necessary to understand & build a C program; it will compile & do something (not much); lines below are written in a text file (say `test.c`)
- First few things to notice:
  - ▹ Program begins with the inclusion of header files (`*.h`), here `stdio.h`; Symbol # indicates that this command drives pre-processor; header files contain function definitions (see next)
  - ▹ A function implementation `main` in the source file (`*.c`)
- Note that reserved names[1] are marked using the blue color

```c
#include<stdio.h>                //Header file

int main(void) {                 //Function implementation
   printf("Hello World\n");
   return 0;
}
```

[1] You can not use these words for any other purpose, their meaning is already defined.

Functions & `main` – mother of all functions

- Any C program will be structured as a set of functions; these eventually receive some data as input, calculate something and then produce an output
- In any C program there must be at least one function called `main()`; this is required to initiate the program execution by clearly identifying the address of the first instruction to be run once the program is loaded into main system memory
- All other functions will be called from this function; in our example we see that in `main` we call `printf()` in line 4

```
1 #include<stdio.h>
2
3 int main(void) {                //Mother of all functions
4    printf("Hello World\n");
5    return 0;
6 }
```

Functions in C – no arguments

- In "Hello World" example, function `main()` has 3 ingredients:
  - function arguments – specified between parenthesis; these are used to "pass" data values from the callee to called function; function can access & process data coming from the outside
  - function name – used to identify given function among others
  - result of a function – **a single value** that a function can produce during execution and return to callee function
- Argument(s) & return value are not mandatory in a function!
  - If there is nothing to pass or return, we use reserved keyword `void`

```c
#include<stdio.h>

int main(void) {     //No argument passed to main
   printf("Hello World\n");
   return 0;
}
```

- Here function `main()` does not take any arguments (`void`)

Functions in C – with argument

- We see that the function `printf()` uses a single argument; multiple arguments would be separated using a coma (,) symbol
- The argument for `printf()` is a string array indicated with double quotes `"`: arbitrary number of characters in this array
- The function will "send" this string array to STDIO (short for standard input/output), i.e. the terminal
  - Note `\n` at the end of the array, indicating "*carriage return*" – forces the terminal to next line; name reflects what happened in the old days with typewriters, or printers when, either the paper and/or printer head moved to the next line for new text

```c
#include<stdio.h>

int main(void) {
   printf("Hello World\n"); //One argument passed to printf
   return 0;
}
```

Functions in C – return value

- Function `main()` should return an integer as specified in line 3; done in line 5 with `return 0;` where we set the return value to 0; callee function can receive this value for further processing
- Function `printf()` also returns something: if you look at documentation you will find:

  If successful, the total number of characters written is returned. On failure, a negative number is returned.

- There is probably a very little use of this return value: we decide not to use it! (return value will be lost, but that's ok)

```
1 #include<stdio.h>
2
3 int main(void) {
4     printf("Hello World\n");
5     return 0;       // We go back, here exit execution
6 }
```

Function definitions – where they are stored

- Left program below will not compile! – we use `sqrt`, some kind of math function (we can guess what it does)
- This is because compiler needs precise function definition, that is:
  - Function name (`main`, `printf`, `myfunct`, etc.)
  - The number of function arguments and their data types
  - What the function produces as a result
- When you implement a function you need to do two things: a) provide function definition (header file) & function implementation (source file)
  - Certain functions are already implemented, but you need to provide the right header file: `math.h` for `sqrt()`

```c
// This will not compile
int main(void) {
    sqrt(7*M_PI);
    // What is M_PI?
}
```

```c
// This solves the problem
#include <math.h>  //Header file
int main(void) {
    sqrt(7*M_PI);
}
```

Different functions defined in math.h

Learn to use documentation, example below: list of math functions and `math.h` header file; if you have a C compiler installed, you may find this file somewhere on your HDD!

fx Functions

Trigonometric functions

cos	Compute cosine (function)
sin	Compute sine (function)
tan	Compute tangent (function)
acos	Compute arc cosine (function)
asin	Compute arc sine (function)
atan	Compute arc tangent (function)
atan2	Compute arc tangent with two parameters (function)

Hyperbolic functions

cosh	Compute hyperbolic cosine (function)
sinh	Compute hyperbolic sine (function)
tanh	Compute hyperbolic tangent (function)
acosh C++11	Compute arc hyperbolic cosine (function)
asinh C++11	Compute arc hyperbolic sine (function)
atanh C++11	Compute arc hyperbolic tangent (function)

Exponential and logarithmic functions

exp	Compute exponential function (function)
frexp	Get significand and exponent (function)
ldexp	Generate value from significand and exponent (function)
log	Compute natural logarithm (function)
log10	Compute common logarithm (function)
modf	Break into fractional and integral parts (function)
exp2 C++11	Compute binary exponential function (function)
expm1 C++11	Compute exponential minus one (function)
ilogb C++11	Integer binary logarithm (function)
log1p C++11	Compute logarithm plus one (function)
log2 C++11	Compute binary logarithm (function)
logb C++11	Compute floating-point base logarithm (function)
scalbn C++11	Scale significand using floating-point base exponent (function)
scalbln C++11	Scale significand using floating-point base exponent (long) (function)

Power functions

pow	Raise to power (function)
sqrt	Compute square root (function)
cbrt C++11	Compute cubic root (function)
hypot C++11	Compute hypotenuse (function)

```c
double __cdecl sin(double);
double __cdecl cos(double);
double __cdecl tan(double);
double __cdecl sinh(double);
double __cdecl cosh(double);
double __cdecl tanh(double);
double __cdecl asin(double);
double __cdecl acos(double);
double __cdecl atan(double);
double __cdecl atan2(double, double);
double __cdecl exp(double);
double __cdecl log(double);
double __cdecl log10(double);
double __cdecl pow(double, double);
double __cdecl sqrt(double);
double __cdecl ceil(double);
double __cdecl floor(double);
double __cdecl fabs(double);
double __cdecl ldexp(double, int);
double __cdecl frexp(double, int*);
double __cdecl modf(double, double*);
double __cdecl fmod(double, double);

double __cdecl _hypot(double, double);
double __cdecl _j0(double);
double __cdecl _j1(double);
double __cdecl _jn(int, double);
double __cdecl _y0(double);
double __cdecl _y1(double);
double __cdecl _yn(int, double);
```

Building more complex SW with few source files

- Programs could contain millions of lines of codes
- You don't want to put all these code lines into a single text file
  - Such file will not only difficult to browse, but compilation times could be long since we have to re-compile everything
  - Compilation of complex/long programs may take hours, if not days, we want to minimize this
  - Often programmers write one function per file
- Assuming multiple C file, how to decide what to compile?
  - Obviously we want to compile only what is need: that is what has been modified & what depends on what has been changed
  - Principle of dependance: any source file could depend on another file (and this can continue), so we need to re-compile all source files that depend on the file that has been modified; further, compilation needs to be performed in a certain order → could be tricky
  - Luckily, modern Integrated Development Environments (IDEs) will automatically detect modified files and file dependancies

3. Data types and variables

Data types in C

- C language defines different data types to minimize the memory usage; when C was invented (70's), memory was scarce
- Data type names are standardized, but the number of bytes allocated for each data type is not! – could be a problem
- Table below summarizes some data types, and ranges assuming certain word bit depth
  - ▹ You should be able to derive the range of a given data type knowing the word depth (at least for integers)

Type	**Bytes**	**Range (min,max)**
`char`	1	– , –
`unsigned char`	1	$0, 2^8 - 1$
`short int`	2	$-2^{15}, 2^{15} - 1$
`unsigned short int`	2	$0, 2^{16} - 1$
`int`	4	$-2^{31}, 2^{31} - 1$
`float`	4	$\pm 3.2 \times 10^{\pm 38}$
`double`	8	$\pm 1.7 \times 10^{\pm 308}$

Variables

- Variable is a symbol that defines a specific place in system memory; it is a symbolic address, since the address is described using a variable name often human readable/understandable
- Variables allow programers to identify easily memory locations
- Any variable **must** be declared before use, by specifying the variable data type & name; HW will use data type information to reserve sufficient amount of memory, and the name to locate that data; more than one variable of the same type can be defined on the same code line, separated with a coma

```
1 void main(void) {
2    int a, a1, a2;          // Multiple vars in the same line
3    long int b, b1, b2;
4    short int c;
5    unsigned int d;
6    char e;
7    float f;
8    double g;
9 }
```

Function `sizeof()`

- Function `sizeof()` returns the size of a given variable (in fact the data type); return value is expressed in number of bytes
- This function is often used when you want to write a portable code (compile & execute for different machine/OS/compiler targets) where these values might change
- You can print variable value using `printf()` function, but you need to add (`%d`) that will format the output
  - ▷ Why do we need to format the print out of the data?

```
1  void main(void) {
2     int a, a1, a2;
3     long int i, b1, b2;
4     short int c;
5     unsigned int d;
6     char e;
7     // %d used to print out memory location as int
8     printf("Size of integer : %d",sizeof(i));
9     printf("Size of character : %d",sizeof(c));
10 }
```

Different formatting when printing

```c
int a;                      /* simple integer type */
long int b;                 /* long integer type */
short int c;                /* short integer type */
unsigned int d;             /* unsigned integer type */
char e;                     /* character type */
float f;                    /* floating point, simple precision */
double g;                   /* floating point, double precision */
a = 1023;
b = 2222;
c = 123;
d = 1234;
e = 'X';
f = 3.14159;
g = 3.1415926535898;
printf("a = %d\n",a);    // decimal
printf("a = %o\n",a);    // octal
printf("a = %x\n",a);    // hex
printf("b = %ld\n",b);
printf("c = %d\n",c);
printf("d = %u\n",d);
```

Continued on the next page

Different formatting when printing (contd.)

```c
printf("e=%c\n",e);
printf("f=%f\n",f);
printf("g=%f\n",g);
printf("a=%d\n",a);
printf("a=%7d\n",a);
printf("a=%-7d\n",a);
printf("f=%f\n",f);       // unformatted
printf("f=%12f\n",f);     // 12 digits
printf("f=%12.3f\n",f);   // 3 for real
printf("f=%12.5f\n",f);   // 5 for real
printf("f=%-12.5f\n",f); // (-) left align
```

```
a = 1023
a = 1777
a = 3ff
b = 2222
c = 123
d = 1234
e = X
f = 3.141590
g = 3.141593
a = 1023
a =    1023
a = 1023
f = 3.141590
f =     3.141590
f =        3.142
f =      3.14159
f = 3.14159
```

Screenshot of the right shows the printout in the terminal

Variables & memory management

- A variable will occupy a place in the memory leaving less memory space for other variables
- Due to sequential nature of computing machines (Turing model of computation) computers create lot of temporary values that can be forgotten as soon as they are generated & have been used
- Even with large memory capacities, if we don't have a mechanism that will throw away unused data, we will quickly run out of memory and compromise the execution of the program
- We don't want programmers to deal with variable management, we want them to focus on algorithm and implementation and not how to keep the memory clean – we look for an indirect (implicit) way of managing memory space
- Life of a variable will be coupled to the life of a function – variables are created when we enter the function, and will be erased when we leave the function

Variable types

- Having in mind the above we define two types of variables:
  - Local variables – life of a local variable is linked to the life of a function that declared this variable; when leaving the function (after `return();`) the variable disappears, memory space is freed
  - Global variables – alive as long as a program lives; all program functions will be able to access them
- Structured programming approach teaches us to avoid usage of global variables at any cost[2]
  - Variables are meant to be modified, global ones too
  - Global vars can be modified by any of the functions in the program
  - For complex problems (many function with intertwined calls) it becomes quickly very hard to keep track of which function is doing what to a variable at what point in time → excellent way to introduce bugs and produce hardly readably code

[2] Not all the features of a programming language should be used!

Example of global & local variables

```
1  #include<stdio.h>
2
3  void func(void);      // function definition
4  int i=4;              // global var defined before function main
5
6  void main(void)
7  {
8      i++;              // use global variable
9      func();           // call a function
10 }
11
12 void func(void)
13 {
14     int j=10;        // local var
15     j++;             // use of a local variable
16     return();        // local var j disappears here
17 }
```

Local variables and scope

- In C it is possible to use the same variable name within a function!
- Compiler understands the difference between the two (or more) variables having the same name, **only if** they are defined in two different blocks of statements identified using curly braces { } (otherwise it will issue a redefinition error)
- Possible thanks to scope of a variable linked to a block or the region in the program where variable is declared, defined, and used

```c
void main(void) {
   int i=4;
   int j=10;
   i++;                          // increments i
   if (j > 0) {
      printf("i is %d\n",i);     // i above is visible here
   }
   if (j > 0) {
      int i=100;                 // local i to this block of code
      printf("i is %d\n",i);
   }                             // this i (100) dies here
   printf("i is %d\n",i);        // i=5 since global was 4
}
```

Global, local variables and scope

- Same reasoning is applied to global variables

```
1  int a;                    // Global variable a
2
3  void f(void)
4  {
5     float a;               // different from global a
6     float b;
7
8     a = 0.0;                /* sets a declared in this block to 0.0 */
9                             /* global a is untouched */
10    {
11       int a;               /// and yet another different a var
12       a = 2;               /* outer a is still set to 0.0 */
13    }
14    b = a;                  /* sets b to 0.0 */
15 }
16 /* global a is untouched by anything done in f() */
```

- Attention: this can be used to write hard-to-read programs
- Check *"International Obfuscated C Code Contest"* http://ioccc.org

Variable storage in memory – the stack

- All variables are stored in a specific memory zone called stack
- The capacity of the stack is fixed at compile time by the compiler, typically from few kB to few dozens of kB; this choice comes from the fact that functions will not often require a lot of memory; some of them will of course need more than few dozens (or hundreds) of kB, we will see how to handle these cases later
- Stack of a function is created upon the function call; when we define a variable, HW will allocate the appropriate amount of memory (because of different data types) in the stack
- When we leave a function, HW will "liberate" the stack, data in the stack is erased & memory freed for other usage
- In this approach HW controls the memory in automated way – computation is structured in functions, they use temporary data (local vars) that are erased when we are done, we keep only what is needed (return value)

Variable storage in memory

- The total amount of memory used by stacks will grow when we call functions in nested fashion, i.e., when one function calls another function, that calls another function, and so on:

$$\mathtt{f}_1(\mathtt{f}_2(\mathtt{f}_3(\ldots(\mathtt{f}_\mathtt{n}))))$$

- When we call functions in a nested way, we need to keep in memory all stacks of callee functions before; in the above $(n-1)$ stacks need to be kept when executing f_n; thus if n is huge, we can run out of memory even if the memory capacity is huge
- In normal programs this is less likely to occur, but beware of recursive functions (those that call themselves) and convergence condition to step out from recursion
  - ▹ Assuming 16GB memory & 32kB stack what is the order of the magnitude of the number of nested calls that you could make before saturating the memory?[3]

[3] Make the assumption that the amount of memory can not be extended in any way.

Variables organization in memory

- Different data types (`unsigned char`, `int`, `float` etc.) need different amount of memory (one variable stores one value)
- Memory is typically addressed by byte – that is one memory address points to a single byte of data
- So one byte of data is the atomic quantity, you can not address less than that (SW could of course extract every single bit)
- Boolean TRUE and FALSE, while binary, will still be encoded using 1 byte of data
- Different variables will be stored sequentially in the memory one after the other
- First variable `a` will be stored in the first address of the stack `Add`
- Consecutive data will be stored on a first free address that can be calculated as: `Add + sizeoff(a)`

Variables organization in memory – example

- Following variables have been declared and initialized

```c
void main(void) {
   unsigned char a=0, b=255;          // var a set to 0
   int           c=1357;
   unsigned char d=45;
}
```

- Memory layout assuming stack starts at `0x00000` is given as:

0x00000	0	0	unsigned char a;
0x00001	0xFF	255	unsigned char b;
0x00002	0x4d	1357	int c;
0x00003	0x05		
0x00004	0x00		
0x00005	0x00		
0x00006	0x2D	45	unsigned char d;

- What is the size of the address space for the example above? What is the maximum memory capacity? Can you explain hex encoding for var `c`?

Stack buffer overflow

Stack doesn't contain only data, but also the address of the callee function essential for safe return; if this address is lost, HW is lost!

In software, a stack buffer overflow or stack buffer overrun occurs when a program writes to a memory address on the program's call stack outside of the intended data structure, which is usually a fixed-length buffer. Stack buffer overflow bugs are caused when a program writes more data to a buffer located on the stack than what is actually allocated for that buffer. This almost always results in corruption of adjacent data on the stack, and in cases where the overflow was triggered by mistake, will often cause the program to crash or operate incorrectly. Stack buffer overflow is a type of the more general programming malfunction known as buffer overflow (or buffer overrun).

Symbolic constants & macros

- We sometimes need to use parameters, constant throughout the whole program; don't use global variables to do that
- Better approach is to use symbolic constants – a very powerful & efficient mechanism for SW parametrization; these use pre-processor directives (indicated using `#define`) for a simple text substitution at compile time

```
#define Pi  3.14  // Some constant
```

- In the above every occurrence of `Pi` will be replaced with 3.14
- Symbolic constants don't use memory!
  - ▹ Can you explain why and where they are stored?
- One can do more → conditional operations using pre-processor directives – macros – they look like functions, but they are not, since once again we deal with text substitution only

```
#define MAX(A,B)   ((A)>(B)?(A):(B))        // Max macro
```

Example of symbolic constants & macros

```c
#include<stdio.h>

#define START  0  /* Starting point of loop */
#define ENDING 9  /* Ending point of loop  */
#define MAX(A,B)  ((A)>(B)?(A):(B))  /* Max macro definition */
#define MIN(A,B)  ((A)>(B)?(B):(A))  /* Min macro definition */

void main(void) {
   int index,mn,mx;
   int count = 5;
   for (index = START;index <= ENDING;index++) {
       mx = MAX(index,count);
       mn = MIN(index,count);
       printf("Max is %d and min is %d\n",mx,mn);
    }
}
```

- Can you explain the difference between normal C functions and macros we introduced above?

How to do operations on "apples & peaches"

- C is strongly typed language – data types need to match; this is not always possible, what do you do if you need to compute:
  `float c = int a / float b`
  and avoid eventual compilation errors or incomplete results
- Assignment operator above requires same left & right data types
- To transform one data type to another, use destination data type between parenthesis placed before the expression – type cast [4]

```
1 void main(void) {
2    int sum = 17, count = 5;
3    double mean;
4
5    mean = (double) sum / count;      //this is explicit typecast
6    printf("Mean : %f\n", mean );
7 }
```

[4] Causes data movements & conversions in the CPU (int & float ALU are not the same!)

Boolean variables and operators

- In C there is **no** Boolean data type that will use a single bit of addressable memory; minimum data size one can use is at least one byte wide
- However there are predefined Boolean values:
  - FALSE – any integer value set to 0 (valid for any data type), and
  - TRUE – anything different from 0; thus TRUE is not necessarily a binary 1 at LSB position!
- As for the operators, there are two different classes of Booleans:
  - Boolean logic operators – manipulate the above defined Boolean values of `TRUE` & `FALSE` as in standard Boolean algebra with traditional Boolean operators `AND`, `OR`, `NOT`
  - Bitwise operators – manipulate variables at bit-level (for the same weight) with traditional Boolean operators

Boolean logical operators

- Represented using double characters, so &&, || for AND and OR (remember that bit-wise operators are single character wide: &, |)
- Bitwise operators have higher precedence over booleans

```
1  #include <stdio.h>
2  void main(void ) {
3     int a = 5;
4     int b = 20;
5     int c ;
6     if ( a && b ) printf("Line 1 - Condition is true\n" );
7     if ( a || b ) printf("Line 2 - Condition is true\n" );
8     // change the value of a and b
9     a = 0;
10    b = 10;
11    if ( a && b ) {printf("Line 3 - Condition is true\n" );}
12       else {printf("Line 3 - Condition is not true\n" );}
13
14    if ( !(a && b) ) {printf("Line 4 - Condition is true\n" );}
15 }
```

- Line 1, 2 & 4 will be printed, but not Line 3. Can you show why?

Boolean bitwise operators

- Can be applied on different (integer) data types
- Bitwise operators – manipulate memory at the bit-level:
  - (~) **bitwise negation** (unary) – flip 0 to 1 and 1 to 0 throughout
  - (&) **AND** (|) **OR** and (^) **exclusive OR** (XOR)
  - >> / << Right/Left Shift of the source operand by 1 bit (destination is the same as source)
    - The above is equivalent to multiply / divide by power 2 – Why?
- In the example:

```
unsigned char a=195;
unsigned char b=87;
unsigned char c;
c=a&b;

printf("Result c: %d\n", c);
```

`printf` will show `c=01000011` in decimal

- Can you show why?

4. Control structures

Control of the instruction flow

- Control statements allow conditional execution of instructions, we have already used execution of mutually exclusive sequences of instructions with `if ... then ... else`
- Different forms are given below; `<expression>` evaluates to a Boolean value of `TRUE` xor `FALSE` as defined previously;
  - ▹ Can you explain "xor" in this sentence?

```
if (<expression>) <statement> // Simple form with no {}'s or else clause

if (<expression>) {           // Multiple statements use curly braces
   <statements>
}

if (<expression>) {         // With else and multiple statements
   <statements>
   } else {
     <statements>
   }
```

- See next slide for other instructions that alter the execution flow

Switch statements

- To allow more compact and readable code when a single variable could take many different values we should favor `switch` rather then a series of `if`

```
switch (<expression>) {
   case <const-expression-1>:
      <statement>
      break;
   case <const-expression-2>:
      <statement>
      break;
   // exp.3, expr4 run same statements
   case <const-expression-3>:
   case <const-expression-4>:
      <statement>
      break;
   default:    // optional
      <statement>
}
```

Example

- Find the largest number among three numbers

```c
#include <stdio.h>

int main() {
  double n1, n2, n3;

  printf("Enter three different numbers: ");
  scanf("%lf %lf %lf", &n1, &n2, &n3);

  // if n1 is greater than both n2 and n3, n1 is the largest
  if (n1 >= n2 && n1 >= n3)
    printf("%.2f is the largest number.", n1);

  // if n2 is greater than both n1 and n3, n2 is the largest
  if (n2 >= n1 && n2 >= n3)
    printf("%.2f is the largest number.", n2);

  // if n3 is greater than both n1 and n2, n3 is the largest
  if (n3 >= n1 && n3 >= n2)
    printf("%.2f is the largest number.", n3);

  return 0;
}
```

Example: another approach

```c
#include <stdio.h>

int main() {
  double n1, n2, n3;

  printf("Enter three numbers: ");
  scanf("%lf %lf %lf", &n1, &n2, &n3);

  // if n1 is greater than both n2 and n3, n1 is the largest
  if (n1 >= n2 && n1 >= n3)
    printf("%.2lf is the largest number.", n1);

  // if n2 is greater than both n1 and n3, n2 is the largest
  else if  (n2 >= n1 && n2 >= n3)
    printf("%.2lf is the largest number.", n2);

  // if both above conditions are false, n3 is the largest
  else
    printf("%.2lf is the largest number.", n3);

  return 0;
}
```

Example: yet another approach

```c
#include <stdio.h>
int main() {
  double n1, n2, n3;

  printf("Enter three numbers: ");
  scanf("%lf %lf %lf", &n1, &n2, &n3);

  if (n1 >= n2) {// outer if statement
    // inner if...else
    if (n1 >= n3)
      printf("%.2lf is the largest number.", n1);
         else printf("%.2lf is the largest number.", n3);
  }

  else {// outer else statement
    // inner if...else
    if (n2 >= n3)
      printf("%.2lf is the largest number.", n2);
         else printf("%.2lf is the largest number.", n3);
  }
  return 0;
}
```

Example: simple calculator

```c
#include <stdio.h>

int main() {
  char op;
  double first, second;
  printf("Enter an operator (+, -, *, /): ");
  scanf("%c", &op);
  printf("Enter two operands: ");
  scanf("%lf %lf", &first, &second);

  switch (op) {
    case '+':
      printf("%.1lf + %.1lf = %.1lf", first, second, first + second);
      break;
    case '-':
      printf("%.1lf - %.1lf = %.1lf", first, second, first - second);
      break;
    case '*':
      printf("%.1lf * %.1lf = %.1lf", first, second, first * second);
      break;
    case '/':
      printf("%.1lf / %.1lf = %.1lf", first, second, first / second);
      break;
    // operator doesn't match any case constant
    default:
      printf("Error! operator is not correct");
}
```

5. Loops

Loops – repeat set of statements

- `while` – evaluates the test expression before every iteration; it can execute zero times if the condition is initially `FALSE`
- `do while` – will always execute at least once
- `for` – initialisation, continuation condition and action

```
while (<expression>) {        // This one could execute zero times
   <statements>
}

do {                          // This one executes at least once
   <statements>
 } while (<expression>)

for (i = 0; i < 10; i++) {    //  Will execute exactly 9 times
   <statements>
 }

for (i = 1; i <= 10; i++) {    //  Will execute exactly 10 times
   <statements>
 }
```

Loops – example of `while` loop

- Do `printf` a certain number of times

```c
#include <stdio.h>

int main()
{
  // Init loop counter, could be anything
  int i = 2;

  // Test expression
  while(i < 10)
  {
    // loop body
    printf( "Hello World\n");

    // update expression
    i++;
  }

  return 0;
}
```

Loops – example of `for` loop

- Compute a sum of all natural numbers until `num`

```c
#include <stdio.h>

int main()
{
    int num, count, sum = 0;
    // Get the input from the keyboard, take it as is for now
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    // for loop terminates when count exceeds num
    for(count = 1; count <= num; ++count)
    {
        sum += count;
    }

    printf("Sum = %d", sum);

    return 0;
}
```

Loops – example of `for` loop

- What do you see at the output?

```c
#include <stdio.h>

int main(){
   int a, b;

   // for loop execution
   for(a = 1, b = 1; a <= 5; a++, b++){
      printf("a: %d b: %d a*b: %d\n", a, b, a*b);
   }

   return 0;
}
```

Forced loop exit

- Loops can be broken, i.e. we force the exit from the loop using another condition and **before** the loop condition becomes FALSE
- In the example below we have an infinite loop! (1 is always true)
- We can exit, because there is a break

```c
#include <stdio.h>

int main () {
   /* local variable definition */
   int a = 10;
   /* while loop execution */
   while( 1 ) { // this is an infinite loop
      printf("value of a: %d\n", a);
      a++;

      if( a > 15)  {
         /* This will terminate the loop */
         break;
      }
   }
return 0;
}
```

Another example

- C Program to reverse a number (e.g. 167 → 761)

```c
#include <stdio.h>

int main() {
  int n, reverse = 0, remainder, original;

  printf("Enter an integer: ");
  scanf("%d", &n);
  original = n;

  while (n != 0) {
    remainder = n % 10;
    reverse = reverse * 10 + remainder;
    n /= 10;
  }
  if (original % 10 == 0) {
       printf("Reversed number = %d", reverse);
       while (original % 10 == 0) {
         printf("0");
         original /= 10;
       }
  } else {
    printf("Reversed number = %d", reverse);
  }
  return 0;
}
```

There is a bug, can you find it?